

Functions Introduction

Learning Objectives

- Define what a function is and explain its purpose in programming
- Identify the components of a function: name, parameters, and return value
- Write simple functions that perform specific tasks

Engage (5-7 minutes)

Imagine you need to calculate the area of 50 different rectangles in a program. Writing the same formula 50 times is tedious and error-prone. A function lets you write the logic once and reuse it whenever needed. Today we explore how functions make code organized, reusable, and easier to maintain.

Explore (10-12 minutes)

Open a code editor and experiment with this idea: write a program that prints “Hello” five times without using any function. Now think about how you could rewrite it so that one block of code handles the greeting, and you just tell it how many times to run. Discuss with a partner what advantages this approach gives you.

Explain (10-12 minutes)

A function is a named block of code that performs a specific task. Functions have a name, can accept inputs called parameters, and can return an output. In most languages, you define a function using keywords like `def`, `function`, or `func`, then call it by its name whenever you need it. Functions follow the DRY principle—Don’t Repeat Yourself—by encapsulating reusable logic.

Elaborate (8-10 minutes)

Write a function called `calculate_area` that takes `length` and `width` as parameters and returns the area. Then write a function called `greet_student` that takes a name and prints a personalized greeting. Call each function at least three times with different values. Consider: what happens if you forget to return a value? What if you pass the wrong number of arguments?

Evaluate (5-8 minutes)

Create a function `convert_temperature` that converts Celsius to Fahrenheit. Write a brief reflection explaining why using functions is better than writing the same code repeatedly, and describe one real-world scenario where a function would be useful in a school management system.